

Introduction to C Programming

What is C?

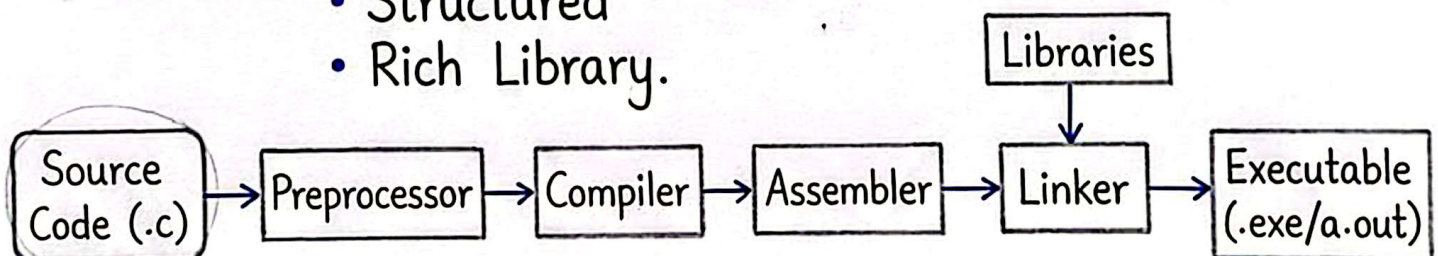
C is a general-purpose, procedural computer programming language supporting structured programming, lexical variable scope, and recursion, with a static type system. It was developed in the early 1970s by **Dennis Ritchie** at **Bell Labs**.

It is widely used for system programming, including developing operating systems (like UNIX), compilers, and embedded systems.

- Features: **Simple**
 - **Efficient**
 - Portable
 - Mid-level
 - Structured
 - Rich Library.

Exam Tip

Remember C is a foundational language for OS development & competitive coding!



Introduction to C Programming

History of C

- C was developed at Bell Labs in 1972.
- It was designed by Dennis Ritchie as an evolution of the B language.
- Derived from an earlier language called BCPL (Basic Combined Programming Language).
- Unix operating system was rewritten in C, demonstrating its power and efficiency.

Key Features of C

- **Procedural Language** - C follows the procedural paradigm, using functions to structure code.
- **Static Type System** - Every variable has a specific data type that is known at compile time.
- **Low-Level Access** - Allows manipulation of bits, bytes, and addresses for system-level programming.
- **Fast & Efficient** - C code is fast due to minimal runtime overhead, making it suitable for performance-critical applications.
- **Powerful Standard Library** - C has a rich set of built-in functions for performing a variety of common tasks.
- **Portability** - C programs can be compiled and run on many different platforms with little or no modification.

Introduction to C Programming

Basic Syntax of C

A C program consists of various components structured in a specific way. A simple C program has the following elements:

- **#include Statements** – Include standard library header files.
(e.g. `#include<stdio.h>`)
- **int main() Function** – The starting point of the program.
- **Braces { }** – Define the beginning and end of a function/block of code.
- **Statements** – Individual instructions end with a semicolon (;).
- **Comments** – Used for adding notes, ignored by the compiler.
(e.g. `// This is a comment`).
- **Return Statement** – Ends the `main()` function and returns a value to the caller.
- **Semicolons** – Every statement ends with a semicolon (;).

```
#include<stdio.h>  → #1 #include Statement
// Include standard library → #2 main() Function
int main() {
    // Main function starts → #3 Comment
    printf("Hello, World!\n"); → #4 Statement
    return 0; // Return 0 to the caller
} → #5 Return Statement
```


Introduction to C Programming

Variables and Data Types

Variables in C must be declared before they can be used. A variable declaration specifies a data type and a name for the variable. Here are some common data types in C:

Data Type	Description	Size	Example
int	Integer type	4 bytes	int age = 25;
float	Floating-point number	4 bytes	float weight = 65.5f;
char	Single character	1 byte	char grade = "A";
double	Double precision floating-point.	8 bytes	double pi = 3.14159;
void	No value (used with functions)	—	—
_Bool	Boolean (true/false) value.	1 byte	_Bool isHappy = 1;



Note:

C is case-sensitive. This means "Age", "AGE", and "age" would be considered different variables.

Variable Declaration

Syntax: `data_type variable_name = value;`

Example: `int age = 25;`



Helpful Tip:

Use meaningful variable names. E.g., use "height" instead of "h" for better readability.

Introduction to C Programming

Input and Output in C

In C, input and output operations are carried out using standard library functions. Here are the primary functions used:

💡 printf() Function

Usage: `printf(format_string, arguments);`

- Format specifiers begin with % to represent variable types within the format string.

Format Specifier	Integer	Example
%d	Floating-point	<code>printf("D:t", %s\n", name);</code>
%f	String	<code>printf("%d", &age);</code>

↳ Example: `int age = 25; float height = 5.9;`
`printf("Age: %d. Height: %.1f feet.\n", age, height);`

Output: Age: 25, Height: 5.9 feet.

💡 scanf() Function

Usage: `scanf(format_string, &variables);`

```
#include<stdio.h>
```

```
// Include standard library
```

```
int main() {
```

```
    int age; // Declare an integer variable
```

```
    printf("Enter your age: "); // Output prompt
```

```
    scanf("%d", &age);
```

```
    printf("You are %d years old.\n", age; // Output result
```

```
    return 0;
```

```
}
```

Output Enter your age: 25

You are 25 years old.

Exam Tip

Remember to use &" before variable names in "scanf" to provide the address of the variable for storing the input value.

Introduction to C Programming

Operators in C

C provides a variety of operators for performing operations on variables and values. Operators in C are categorized as follows:

Arithmetic Operators

Symbol	Description	Example
+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
%	Modulus	$a \% b$

Relational Operators

Symbol	Description	Example
==	Equal to	$a == b$
!=	Not equal to	$a != b$
>	Greater than	$a > b$
<	Less than	$a < b$
>=	Greater than or equal to	$a >= b$

Logical Operators

Symbol	Description	Example
&&	AND	$(a > 0) \&\& (b > 0)$
	OR	$(a > 0) (b < 0)$
!	NOT	$!(n == 0)$

Example:

```
int a = 10, b = 5;
int sum = a + b;
if ((a > 0) && (b < 10)) {
    sum += 5;
    printf("Sum: %d\n", sum); // Outputs: Sum: 20
}
Outputs: Sum: 20
```

Assignment Operators

Symbol	Description	Example
=	Assign	$a = 5$
+=	Add and assign	$a += 3 (a = a + 3)$
-=	Subtract assign	$a -= 2 (a = a - 2)$
*=	Multiply and assign	$a *= 4 (a = a * 4)$
/=	Divide and assign	$a /= 2 (a = a / 2)$

@mastercode.sagar

Introduction to C Programming

Control Structures in C

Control structures allow you to manage the flow of a program. They include conditional statements and loops to control the execution of code blocks:

💡 Conditional Statements

Conditional statements are used for decision-making.

if Statement	
Usage: <code>if (condition) { /* code */ }</code>	<code>if (x > 0) { printf("Positive\n"); }</code>
if-else Statement	
Usage: <code>if (condition) { /* code if true */ /* code if false */ }</code>	<code>if (x > 0) { printf("Positive\n"); }</code>
if-else if-else Statement	
Usage: <code>if (condition1) { /* code if condition1 is true */ /* code if condition2 is true */ /* code if all conditions are false */ }</code>	<code>if (x > 0) { printf("Negative\n"); } else { printf("Zero\n"); }</code>

💡 Loops:

Loops allow repetitive execution of a block of code. The primary loops in C.

• for Loop	Used to execute a block of code a specific number of times.
• while Loop	Executes a block of code as long as a condition is true.
• do-while Loop	Executes a block of code once, then repeats as long as condition is true.

💡 Note:

Control structures are fundamental for writing flexible and efficient programs.

Introduction to C Programming

Loops in C

Loops are used to execute a block of code repeatedly. In C, there are three primary types of loops:

- for Loop
- for Loop
- while Loop

💡 for Loop

Usage: for (initialization; condition; increment) {

• Example

Output: 0 1 2 3 4

```
for (int i = 0; i < 5; i++)
    printf("%d", i);
}
```

Output: 0 1 2 3 4

💡 while Loop

Usage: while (condition) {

• Example:

Output: 0 1 2 3 4

```
int i = 0;
while (i < 5) {
    printf("%d", i);
    i++;
}
```

💡 do-while Loop

Loops allow repetitive loop, the do-while loop guarantees at least one iteration of the code block.

• Example:

Output: 0 1 2 3 4

```
int i = 0;
do {
    printf("%d", i);
    i++;
} while (i < 5);
```

💡 Tip

Unlike the while loop, the do-while loop guarantees at least one iteration of the code block.

Introduction to C Programming

Functions in C

Functions are used to break down programs into smaller, reusable pieces of code. They help to organize and manage code efficiently:

💡 Syntax:

```
return_type function_name(parameter1, parameter2, ...) {
    return value;
} /* code */
```

A function in C has the following components:

• Return Type:	Specifies the type of value the function returns (eg., int, void)
• Function Name:	The identifier used to call the function (e.g, sum)
• Parameters:	Inputs to the function, specified inside parentheses (optional)
• Return Value:	The value the function sends back to the caller (optional)

💡 Example:

Usage: while (condition) {	int i = 0;
• Example:	while (i < 5) {
Output: 0 1 2 3 4	printf('%d', i);
	i++;
	} while (i < 5);

💡 do-while Loop:

Usage: do { /* code */	int i = 0;
while (condition);	printf('%d', i);
	i++;
Output: 0 1 2 3 4	} while (i < 5);

💡 Tip

Unlike the while loop, the do-while loop guarantees at least one iteration of the code block.

Introduction to C Programming

Arrays in C

Arrays are used to store multiple values of the same data type. An array is a collection of elements that can be accessed using an index.

💡 Syntax:

```
data_type array_name[size]; /* code */
```

A function in C has the following components:

• Return Type:	Specifies the type of value the function returns (eg, int, float)
• array_name:	The identifier used to call the array
• size:	Inputs to the function, specified inside parentheses (optional)

💡 Example:

Syntax: `data_type array_name[size] = {value1, value2, ...};`

```
int numbers[5] = {10, 20, 30, 40, 50};
```

Output: 0 1 2 3 4

10	20	30	40	50	50
0	1	2	3	3	4

💡 Example:

Usage: `data_type array_name[size] = {value1, value2, ...};`

```
int numbers[5] = {10, 20, 30, 40, 50};
```

```
printf('Second element: %d\n', numbers[1]); // Access the second element.
```

Output: 0 1 2 3 4

💡 Note:

Array indexing starts at 0, so the first element is at index 0, the second at index 1, and so on.

Introduction to C Programming

More on Arrays

Arrays in C can be manipulated in various ways. Here are some common operations and concepts related to arrays:

💡 Syntax:

Array Initialization: Arrays can be initialized at the time of declaration.

◦ **Syntax:** `data_type array_name[size] = { value1, value2, ... };`

• A function in C has the following components:

◦ **data_type:** The data type of the elements (e.g., int, float)

◦ **array_name:** The identifier used to call the array

💡 Example:

Syntax: `data_type array_name[size] = { value1, value2, ... };`

```
int numbers[5] = { 10, 20, 30, 40, 50};
```

// C++ See: int #1 value = 87; index # 0

Output: 0 1 2 3 4

💡 Array Length:

The length (number of elements) of an array can be determined using:

```
length = sizeof(array_name) / sizeof(data_type);
```

```
int numbers[5] = { 10, 20, 30, 40, 50};
```

```
int length = sizeof(numbers) / sizeof( numbers[0]);
```

```
printf("Array length: %d\n", length);
```

Output: Array length: 5

💡 Note:

Be careful when accessing arrays. Accessing an index out of bounds (e.g., `numbers[5]`) can lead to undefined behavior or crashes.

Introduction to C Programming

Pointers in C

Pointers are variables that store the memory address of another variable. They are used for dynamic memory allocation, arrays, and functions.

💡 Syntax:

```
data_type *pointer_name; /* code */
```

- A function in pointer selection various ways:

- **Syntax:** The data type of the variable the pointer (e.g., int, float)

- *****: Indicates the indicates variable a pointer?

- **pointer_name:** The identifier used to call the pointer

💡 Example:

```
int num = 10; ] = Pointer for ie & num
int *ptr = &num; // Pointer to num
printf((Value of num: %d\n., num);
printf((Value pointed to by ptr: %d\n, *ptr);
```

Output: value of num: 10

Value pointed to by ptr: 10

Pointer and Address-of Operator

The address-of operator (&) gives the memory address of a variable.

Pointer:

```
int numbers[5] = {10, 20, 30, 40};
ptr memory numbers[1]; // true, and.
```

Output: Array length: 5

💡 Dereferencing Pointer

The dereferencing operator (*) accesses the value at the pointers address.

```
ptr = *tender(array_namer)
atemer{stie_ptrte;
printf("\Array length, %d\n, length);
```

Output: Array length: 5

Introduction to C Programming

Dynamic Memory Allocation

Dynamic memory allocation allows programs to request and manage memory while the program is running. In C, we use the `stdlib.h` library functions to allocate and free memory dynamically:

💡 Syntax:

malloc() Allocates a specified number of bytes and returns a pointer to the allocated memory.

• Syntax: `pointer = (data_type*) malloc(size_in_bytes);`

• A function in C has the following components:

malloc() `int * arr;`

`arr = (int*) malloc(5 * sizeof(int));` // Dynamically allocate an array

p. ✎ `if (arr != NULL);`

`arr[0] = 10;`

`printf("First element : %d\n", arr[0]);`

free(arr); // Free dynamically allocated memory

Output: First element: 10

Pointer and Address-of Operator

The address-of operator (`&`) gives the memory address of a variable.

Syntax:

`free(pointer_name);`

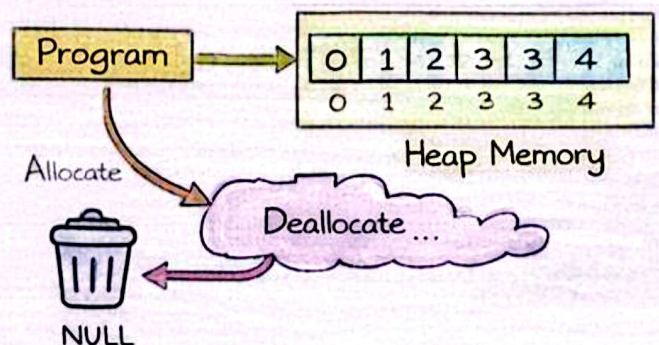
Output: First element: 10

💡 Note:

Failing to free dynamically allocated memory can lead to memory leaks.

💡 Dereferencing Pointer

Frees dynamically allocated memory, releasing it for future use.



Introduction to C Programming

More on Dynamic Memory Allocation

Beyond malloc() and free(), the realloc() function can be used to resize previously allocated memory blocks.

💡 Syntax:

```
pointer = realloc(pointer, new_size); // f code */
```

- A function in C has the following components:

• Syntax: `pointer = (data_type*) malloc(size_in_bytes);`

- A function in C has the following components:

```
int *arr;
arr = (int*) malloc(3 * sizeof(int)); // Allocate an array of 3 integers
if (arr != NULL);
    arr[0] = 10;
    arr[2] = 20;
    arr = realloc(arr, 5 * sizeof(int)); // Resize the array to hold 5 integers
    if (arr != NULL);
        arr[3] = 40;
        arr[4] = 50;
    printf("Elements in the array: ");
```

Output: First element: 10

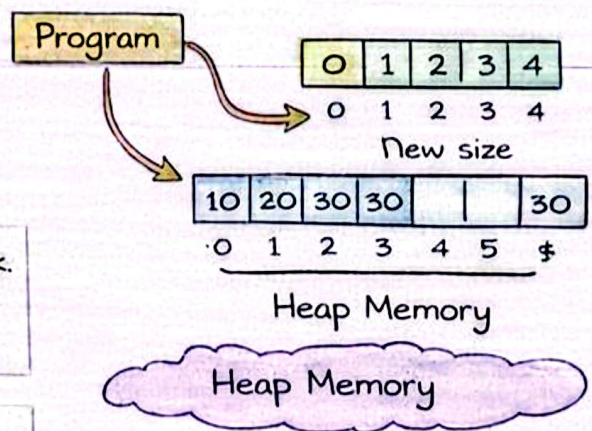
Pointer "arr" points here

Tip: Always check if realloc() successfully reallocated the memory before using it.

Tip: realloc() can be used to grow or shrink an array of dynamically allocated memory to a new size.

💡 Note:

Failing to free dynamically allocated memory can lead to memory leaks.



Introduction to C Programming

More on Dynamic Memory Allocation

Beyond malloc() and free(), the realloc() function can be used to resize previously allocated memory blocks:

💡 if Statement:

- Executes a block of code if a condition is true.

Syntax: `pointer = realloc(pointer, new_size);`

*/ * code */*

- A function in C has the following components:

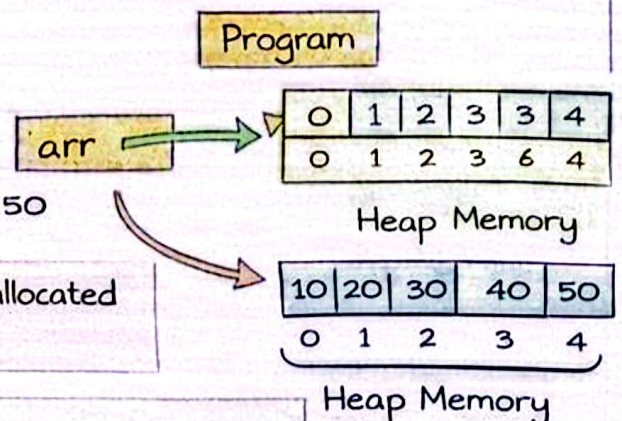
• **Syntax:** `pointer = realloc( pointer, new_size);`

Example:

```
int num = 10;
if (num > 0) { // Allocate an array of 5 integers
    arr[0] = 10;
    arr[2] = 20;
    arr = realloc(arr, 5 * sizeof(int)); // Resize the array to hold 5 integers
    if (arr != NULL);
        arr[3] = 40; ← Pointer "arr" points here
        arr[4] = 50;
    printf("Elements in the array: ");
    for (int i = 0, < 5);
    free(arr);
}
```

Output: Elements in the array: 10 20 30 40 50

Tip: Always check if realloc() successfully reallocated the memory before using it.



💡 Note:

It's important to use proper conditionals to control the flow of a program and handle different situations.

Introduction to C Programming

Examples of Conditionals

Here are three examples illustrating the use of conditional statements in C.

💡 if Statement

- Executes a block of code if a condition is true.

Syntax: `pointer = realloc(pointer, new_size);`

```
int num = 15;
if (num > 0){
    printf("The number is positive.\n");
}
```

Output: The number is positive.

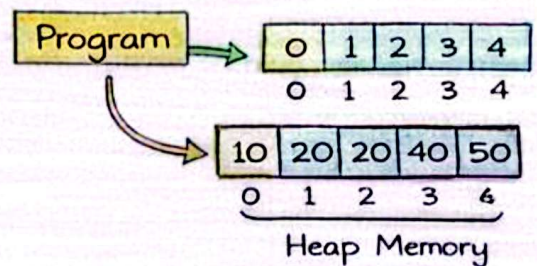
💡 if-else Statement

- Executes one block of code if a condition is true, and another block if it is false.

Syntax:

```
int num = 42;
if (num % 2 == 0){
    printf("The number is even.\n");
} else {
    printf("The number is odd.\n");
}
```

Output: Non positive.



💡 if-else if-else Statement:

- Example: Check whether a number is positive, negative, or zero.

💡 Tip:

Consider all possible conditions to ensure your program behaves correctly under different inputs.

Introduction to C Programming

The switch Statement

Here are three examples illustrating the use of conditional statements in C.

💡 if Statement

- Executes a block of code if a condition is true.

Syntax: `switch(variable).`

`/ f code */`

- A faction name can to print the day number.
- `Switch (variable) {`
`case value1:`
`(f code for value1 }`
`break;`
`}`

Output: The number is positive.

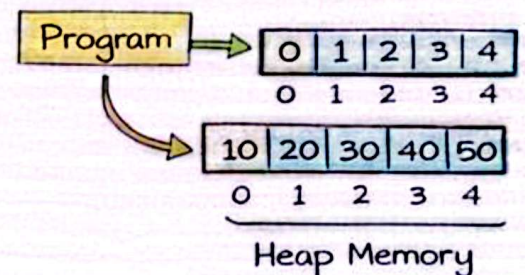
💡 if-else Statement:

- Executes one block of code if a condition is true, and another block if it is false.

Syntax:

```
int num = 42;
if (num % 2 == 0){
    printf("The number is even.\n");
} else {
    printf("Non positive \n");
}
```

Output: Wednesday



Heap Memory

Output: Wednesday

💡 Note:

Its impuder all possible conditions to ensure your program behaves correctly under different inputs.